Why Dynamic Memory?

- Limitations of static memory allocation
- Need for flexible memory management
- Allocation of memory during runtime

Static Memory Allocation

Definition:
Static memory allocation refers to the allocation of memory at compile time, where the size and type of variables are known before program execution.

Key Features:

- Accessible throughout the program
- Fixed size at compile time
- Memory allocated in data segment
- No explicit deallocation required

Dynamic Memory Allocation

Definition:
Dynamic memory allocation refers to allocating memory at runtime, allowing programs to request memory as needed.

Key Features:

- Allocation happens at runtime
- Supports dynamically sized data structures
- Requires manual memory management
- Uses functions from `<stdlib.h>`

Static vs Dynamic Memory Allocation

Feature	Static Memory Allocation	Dynamic Memory Allocation
Allocation Time	Compile-time	Runtime
Memory Size	Fixed	Variable
Lifetime	Entire program	Can change during execution
Storage	Stack / Global	Heap
Flexibility	Limited	High
Initialization	Default or explicit	May be required
Deallocation	Automatic	Manual (`free`)

Stack vs Heap

Stack

- Memory managed automatically
- Smaller in size
- Faster access
- Not flexible
- Scope limited to function/thread

Heap

- Memory managed manually
- Larger in size
- Slower access
- Flexible
- Globally accessible

Dynamic Memory Allocation Functions

Overview:

- `malloc` → Memory allocation
- `calloc` → Contiguous allocation + initialization
- `realloc` → Resize memory
- `free` → Deallocate memory

---

malloc Function

Syntax:

```
int* ptr = (int*) malloc(sizeof(int));
```

Explanation:

- Allocates memory for a single integer
- Returns a pointer to allocated memory

Dynamic Memory Allocation – C Program Example

```
#include <stdio.h>

#include <stdlib.h>



int main() {

    int size;



    // Get the size of the array from the user

    printf("Enter the size of the array: ");
```

```
scanf("%d", &size);

// Dynamically allocate memory for an array
int *dynamicArray = (int *)malloc(size * sizeof(int));

// Check if memory allocation is successful
if (dynamicArray == NULL) {
    fprintf(stderr, "Memory allocation failed.\n");
    return 1;
}

// Input values
printf("Enter %d integers:\n", size);
for (int i = 0; i < size; i++) {
    scanf("%d", &dynamicArray[i]);
}

// Display values
printf("Values stored in the dynamically allocated array:\n");
for (int i = 0; i < size; i++) {
    printf("%d ", dynamicArray[i]);
}
```

```
    // Free memory

    free(dynamicArray);


    return 0;

}
```

---

calloc Function

Syntax:

```
int* ptr = (int*) calloc(n, sizeof(int));
```

Explanation:

- Allocates memory for multiple elements
- Memory is initialized to zero
- Allocates contiguous memory blocks

---

malloc vs calloc

	malloc()	**calloc()**
	Allocates single block	**Allocates multiple blocks**

Takes 1 argument	**Takes 2 arguments**
Faster	**Slower**
Does not initialize memory	**Initializes memory to 0**
May contain garbage values	**Values are zero**

---

realloc Function

Syntax:

```
ptr = realloc(ptr, new_size);
```

Explanation:

- Resizes previously allocated memory
- Can increase or decrease size
- Preserves existing data (up to new size)

---

free Function

Purpose:

- Deallocates memory allocated using malloc/calloc/realloc

Syntax:

```
free(ptr);
```

Memory Leak

Explanation of Memory Leaks

- Unused memory blocks
- Global pointers retaining memory

Causes:

- Failure to free allocated memory
- Overwriting the only reference to allocated memory
- Mismatched allocation and deallocation functions

Common Pitfalls

- Dangling Pointers
  Pointer refers to freed memory
- Double Freeing
  Freeing same memory twice
- Memory Fragmentation
  Inefficient memory usage due to small unused blocks

1. Program using calloc()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size;

    printf("Enter number of elements: ");
    scanf("%d", &size);

    // Allocate memory using calloc (initialized to 0)
    int *arr = (int *)calloc(size, sizeof(int));
```

```
    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter %d values:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Array elements are:\n");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

---

2. Program using realloc()

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int size, newSize;

    printf("Enter initial size: ");
    scanf("%d", &size);

    int *arr = (int *)malloc(size * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    printf("Enter %d elements:\n", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
```

```
    // Resize array
    printf("Enter new size: ");
    scanf("%d", &newSize);

    arr = (int *)realloc(arr, newSize * sizeof(int));

    if (arr == NULL) {
        printf("Reallocation failed\n");
        return 1;
    }

    printf("Enter new elements:\n");
    for (int i = size; i < newSize; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Updated array:\n");
    for (int i = 0; i < newSize; i++) {
        printf("%d ", arr[i]);
    }

    free(arr);
    return 0;
}
```

---

3. Program using free() (Simple Example)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(5 * sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        ptr[i] = i + 1;
    }
```

```
    printf("Before freeing:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);
    }

    free(ptr);   // memory released
    ptr = NULL;  // good practice

    return 0;
}
```

---

4. Pitfalls Programs (Important for Viva)

---

a) Dangling Pointer

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(sizeof(int));

    *ptr = 10;
    free(ptr);

    // Dangling pointer (memory already freed)
    printf("%d", *ptr);  // ❌ Undefined behavior

    return 0;
}
```

---

b) Double Free

```
#include <stdio.h>
#include <stdlib.h>

int main() {
```

```
    int *ptr = (int *)malloc(sizeof(int));

    free(ptr);
    free(ptr);  // ❌ Error: double freeing

    return 0;
}
```

---

c) Memory Leak

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(sizeof(int));

    ptr = NULL;   // ❌ Lost reference (memory leak)

    return 0;
}
```

---

d) Uninitialized Memory (Garbage Values)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(5 * sizeof(int));

    // Not initialized
    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr[i]);  // ❌ Garbage values
    }

    free(ptr);
    return 0;
}
```

Best Practices (Dynamic Memory Allocation)

- Always check for successful memory allocation (NULL check)
- Free allocated memory when no longer needed
- Avoid using uninitialized memory

Consequences of Poor Memory Management

1. Gradual Performance Degradation
    - Program becomes slower over time
2. Increased Resource Consumption
    - More memory and CPU usage
3. Potential Out-of-Memory (OOM) Conditions
    - Program may crash due to insufficient memory
4. Reduced Responsiveness
    - System or application becomes slow
5. Longer Program Execution Times
    - Increased processing delays
6. Unpredictable Behaviour
    - Program may give incorrect or inconsistent results